CS224N: Natural Language Processing with Deep Learning

Lecture 4: Dependency Parsing

Study Notes

Stanford University

Contents

Key Learning Objectives

- Understand explicit linguistic structure in sentences
- Learn how neural networks can determine syntactic structure
- Master dependency parsing concepts and algorithms
- Implement neural dependency parsers

1 Introduction to Syntactic Structure

1.1 Why Sentence Structure Matters

Human communication involves composing words into larger units to convey complex meanings. Understanding how words relate to each other (what modifies what) is essential for:

- Correct interpretation of language
- Resolving ambiguities
- Extracting semantic information
- Building effective NLP systems

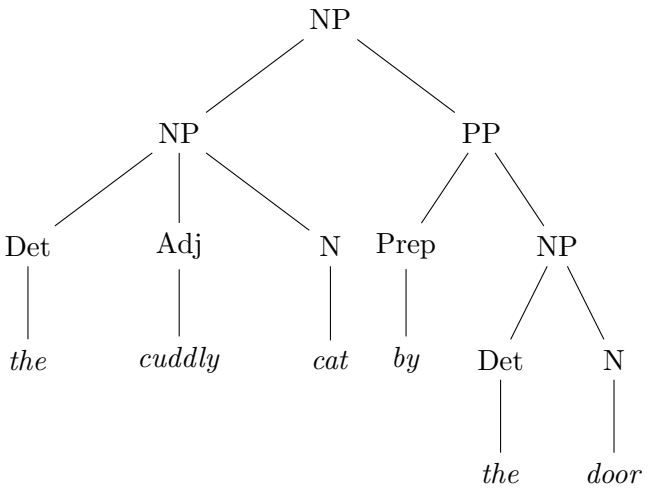
1.2 Two Views of Linguistic Structure

1.2.1 Constituency Grammar (Phrase Structure)

Also known as **Context-Free Grammars (CFGs)**, this approach organizes words into nested constituents.

Example hierarchy:

- Words: *the, cat, cuddly, by, door*
- Phrases: *the cuddly cat, by the door*
- Larger phrases: *the cuddly cat by the door*

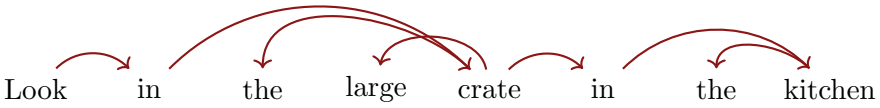


Sample CFG Rules:

$$\begin{aligned}
\text{NP} &\rightarrow \text{Det Adj}^*\text{N (PP)} \\
\text{PP} &\rightarrow \text{Prep NP} \\
\text{VP} &\rightarrow \text{V PP} \\
\text{Det} &\rightarrow \text{the} \mid \text{a} \\
\text{Adj} &\rightarrow \text{large} \mid \text{cuddly}
\end{aligned}$$

1.2.2 Dependency Structure

Dependency structure shows which words depend on (modify, attach to, or are arguments of) other words through directed arrows.



Key Terminology

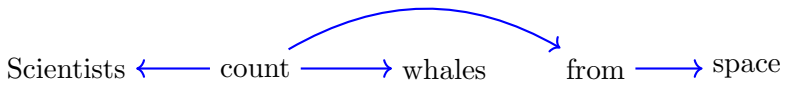
- **Head** (governor, superior): The word that another word depends on
- **Dependent** (modifier, inferior): The word that modifies or attaches to the head
- Dependencies typically form a **tree**: connected, acyclic, single-root graph

2 Ambiguity in Natural Language

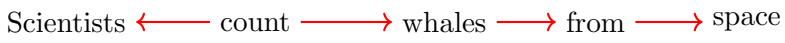
2.1 Prepositional Phrase (PP) Attachment Ambiguity

Example: "Scientists count whales from space"

Interpretation 1: Counting happens from space



Interpretation 2: Whales are from space



2.2 Catalan Numbers and Ambiguity

The number of possible attachment structures grows exponentially with prepositional phrases:

$$C_n = \frac{(2n)!}{(n+1)!n!}$$

# of PPs	# of Readings
1	1
2	2
3	5
4	13
5	27

Example with 4 PPs:
"The board approved its acquisition by Royal Trustco Ltd. of Toronto for $27 a share at its monthly meeting"

This sentence has 13 possible structural interpretations!

2.3 Other Types of Ambiguity

1. **Coordination Scope:** "Shuttle veteran and longtime NASA executive Fred Gregory" (1 or 2 people?)
2. **Adjectival/Adverbial Modifier:** Different attachment points for modifiers
3. **VP Attachment:** Verb phrase boundaries can be ambiguous

3 Dependency Grammar: History and Structure

3.1 Historical Background

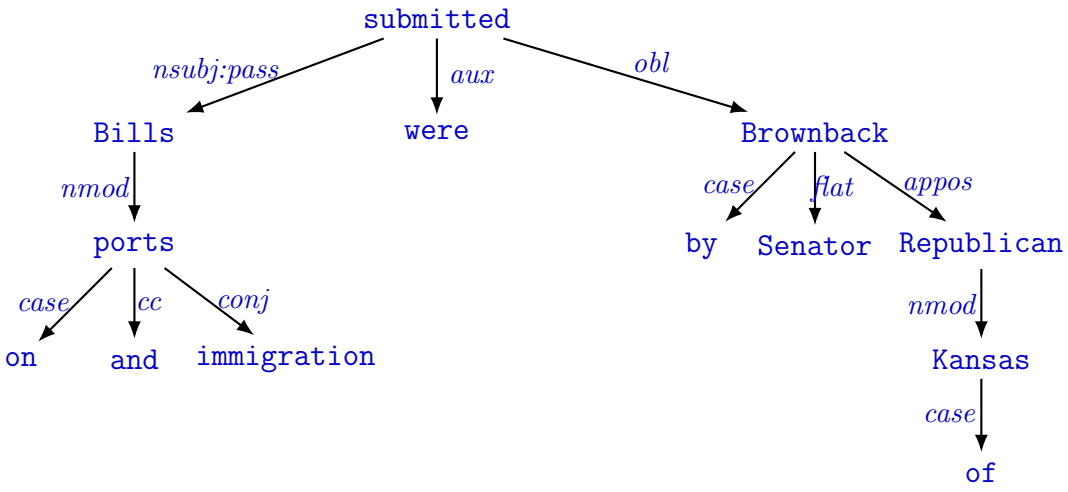
- **Ancient origins:** Pāṇini's grammar (c. 5th century BCE) - first dependency grammar
- **Arabic grammarians:** 1st millennium approaches
- **Modern formalization:** Lucien Tesnière (1959)
- **CFGs are recent:** R.S. Wells (1947), Chomsky (1953)
- **Early NLP:** David Hays built first dependency parser (1962)

Historical Note

Pāṇini's grammar was composed and transmitted **orally** for centuries before being written down - an astounding intellectual achievement!

3.2 Dependency Structure Notation

Example with typed relations:



Common dependency relations:

- `nsubj`: nominal subject
- `obj`: object
- `obl`: oblique (prepositional phrase)
- `det`: determiner
- `amod`: adjectival modifier
- `case`: case marking (prepositions)

3.3 Projectivity

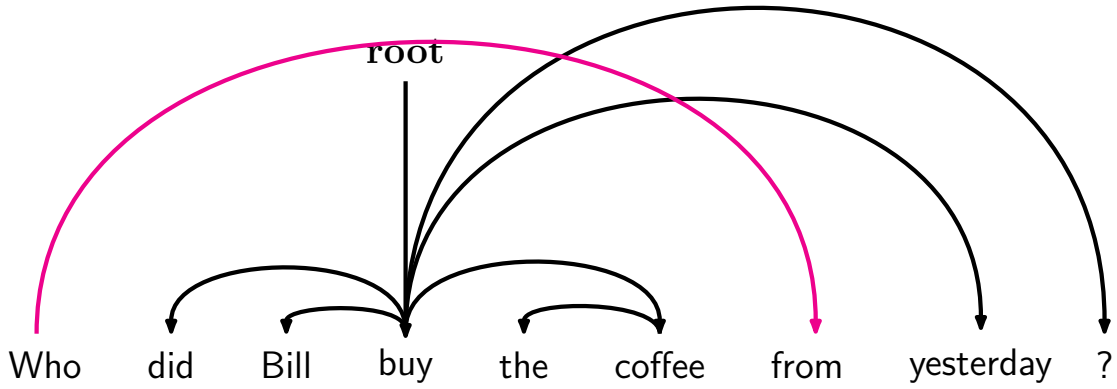
Projective Parse: No crossing dependency arcs when words are in linear order with arcs above.

Non-projective: Crossing arcs can occur (e.g., in questions or free word order languages)

Example of Non-Projective Structure: Most syntactic structure is projective like this, but dependency theory normally does allow non-projective structures to account for displaced constituents

Why allow non-projective dependencies?

- You can't easily get the semantics of certain constructions right without these nonprojective dependencies



4 Dependency Parsing Preferences

Four main sources of information guide dependency parsing:

1. **Bilexical affinities:** "discussion of issues" is plausible
2. **Dependency distance:** Most dependencies are between nearby words
3. **Intervening material:** Dependencies rarely span verbs or punctuation
4. **Valency:** How many dependents does a head typically take?

5 Methods of Dependency Parsing

5.1 Overview of Approaches

1. **Dynamic Programming:** Eisner (1996) - $O(n^3)$ complexity
2. **Graph Algorithms:** MSTParser (McDonald et al. 2005) - $O(n^2)$
3. **Constraint Satisfaction:** Eliminate edges violating constraints
4. **Transition-based:** Greedy, guided by ML classifiers (focus of this lecture)

5.2 Transition-Based Parsing (Nivre 2003)

A greedy discriminative approach using a sequence of bottom-up actions.

5.2.1 Parser Components

Data Structures

- **Stack σ:** Top written to the *right*, starts with ROOT
- **Buffer β:** Top written to the *left*, starts with input sentence
- **Arc set A:** Dependency arcs, starts empty

5.2.2 Basic Transitions

Initial state: $\sigma = [\text{ROOT}]$, $\beta = w_1, \ldots, w_n$, $A = \emptyset$

Transitions:

1. **Shift:** $\langle \sigma, w_i|\beta, A\rangle \Rightarrow \langle \sigma|w_i, \beta, A\rangle$
2. **Left-Arc$_r$:** $\langle \sigma|w_i|w_j, \beta, A\rangle \Rightarrow \langle \sigma|w_j, \beta, A \cup \{r(w_j, w_i)\}\rangle$
3. **Right-Arc$_r$:** $\langle \sigma|w_i|w_j, \beta, A\rangle \Rightarrow \langle \sigma|w_i, \beta, A \cup \{r(w_i, w_j)\}\rangle$

Final state: $\sigma = [w]$, $\beta = \emptyset$

5.2.3 Example: Parsing "I ate fish"

Start: $\sigma = [\text{ROOT}]$ $\beta = [\text{I, ate, fish}]$

Shift: $\sigma = [\text{ROOT, I}]$ $\beta = [\text{ate, fish}]$

Shift: $\sigma = [\text{ROOT, I, ate}]$ $\beta = [\text{fish}]$

Left-Arc: $\sigma = [\text{ROOT, ate}]$ $A = \{\text{nsubj(ate}\rightarrow\text{I)}\}$

Shift: $\sigma = [\text{ROOT, ate, fish}]$ $\beta = [\,]$

Right-Arc: $\sigma = [\text{ROOT, ate}]$ $A = \{..., \text{obj(ate}\rightarrow\text{fish)}\}$

Right-Arc: $\sigma = [\text{ROOT}]$ $A = \{..., \text{root(ROOT}\rightarrow\text{ate)}\}$

- We have left to explain how we choose the next action
  - Answer: Stand back, I know machine learning!
- Each action is predicted by a discriminative classifier (e.g., softmax classifier) over each legal move
  - Max of 3 untyped choices (max of $|R| \times 2 + 1$ when typed)
  - Features: top of stack word, POS; first in buffer word, POS; etc.
- There is NO search (in the simplest form)
  - But you can profitably do a beam search if you wish (slower but better):
    - You keep k good parse prefixes at each time step
- The model's accuracy is *a bit* below the state of the art in dependency parsing, but
- It provides **very fast linear time parsing**, with high accuracy – great for parsing the web

6 Neural Dependency Parsing

6.1 Problems with Traditional Features

Traditional feature-based parsers (MaltParser) used indicator features:

Problems:

- **Sparse:** Individual features rarely observed
- **Incomplete:** Missing combinations not in training data
- **Expensive:** ¿95% of parsing time spent on feature computation

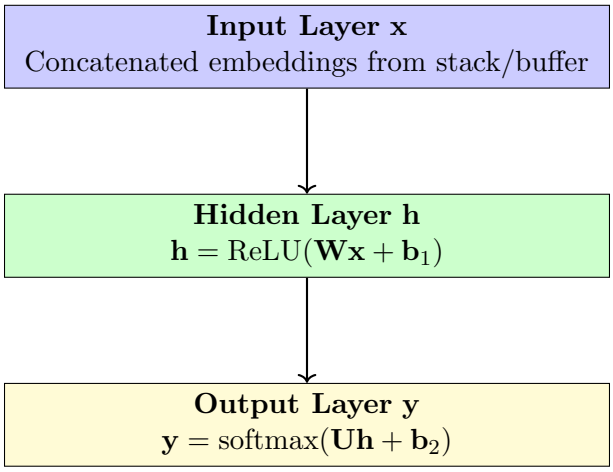
Indicator features: Binary, sparse vectors with dimension $10^6 - 10^7$

6.2 Neural Approach (Chen & Manning 2014)

Key innovations:

1. **Distributed representations:** Dense vectors ($d \approx 1000$) for words, POS tags, and dependency labels
2. **Non-linear classifier:** Deep neural network instead of linear classifiers

6.2.1 Architecture



Softmax over: {SHIFT, LEFT-ARC$_r$, RIGHT-ARC$_r$}

6.2.2 Token Extraction

From the parser configuration, extract tokens from:

- Top 2 words on stack: s_1, s_2
- First word in buffer: b_1
- Left/right children of s_1, s_2: $lc(s_1), rc(s_1), lc(s_2), rc(s_2)$

For each token, concatenate:

- Word embedding
- POS tag embedding
- Dependency label embedding (if applicable)

6.3 Performance Results

Parser	UAS	LAS	Sentences/sec
MaltParser	89.8	87.2	469
MSTParser	91.4	88.1	10
TurboParser	92.3	89.6	8
Chen & Manning 2014	**92.0**	**89.7**	**654**

Key achievement: Comparable accuracy to graph-based parsers with much faster speed!

6.4 Further Developments

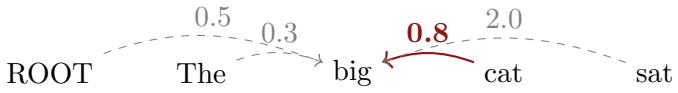
Google's SyntaxNet / Parsey McParseFace (2016):

- Deeper networks with better hyperparameters
- Beam search instead of greedy decoding
- CRF-style global inference
- **Results:** UAS 94.61%, LAS 92.79%

7 Graph-Based Dependency Parsing

7.1 Approach

1. Compute a score for *every possible dependency* for each word (n^2 possibilities)
2. Use contextual representations of word tokens
3. Apply Minimum Spanning Tree (MST) algorithm to find best tree structure



7.2 Dozat & Manning (2017)

Neural graph-based parser with biaffine scoring:

- Uses neural sequence models (discussed in later lectures)
- State-of-the-art accuracy: **UAS 95.74%, LAS 94.08%**
- Slower than transition-based parsers due to $O(n^2)$ scoring
- Available in Stanford's *Stanza* toolkit

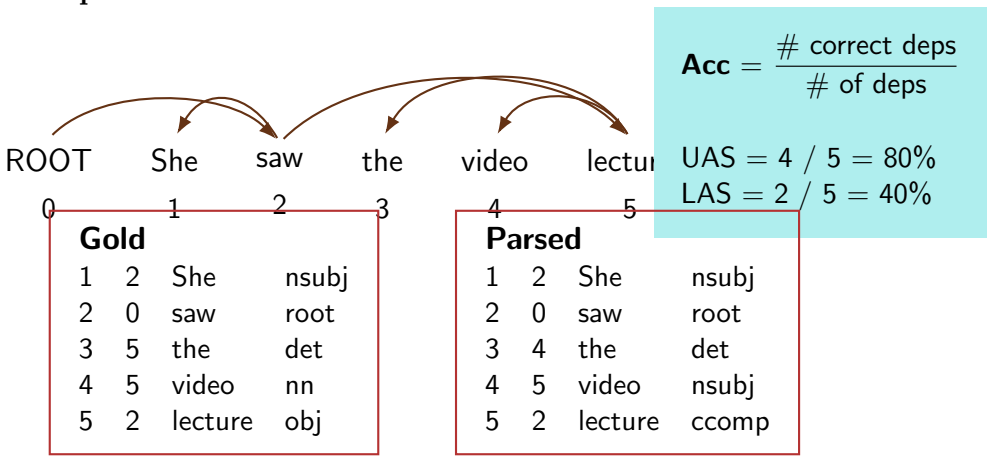
8 Evaluation Metrics

8.1 Dependency Accuracy

Given a gold standard parse and a predicted parse:

- **Unlabeled Attachment Score (UAS):** % of words with correct head
- **Labeled Attachment Score (LAS):** % of words with correct head *and* label

Example:



9 Universal Dependencies

9.1 Modern Treebanks

Universal Dependencies (UD): `http://universaldependencies.org/`

- Cross-linguistic dependency annotation framework
- 100+ languages with consistent annotation
- Enables multilingual NLP research
- Training data for Assignment 2!

9.2 Benefits of Annotated Data

1. **Reusability:** Multiple systems can be built from same resource
2. **Broad coverage:** Real-world language, not just linguist intuitions
3. **Frequency information:** Statistical patterns for ML
4. **Evaluation:** Systematic assessment of NLP systems

10 Practical Applications

10.1 Information Extraction

Example: Protein-protein interaction

Sentence: "KaiC interacts rythmically with SasA and KaiA"

Dependency path: KaiC $\xleftarrow{\text{nsubj}}$ interacts $\xrightarrow{\text{nmod:with}}$ SasA $\xrightarrow{\text{conj:and}}$ KaiA

Extracted interactions:

- KaiC $\leftrightarrow$ SasA
- KaiC $\leftrightarrow$ KaiA

11 Summary

Key Takeaways

1. Dependency parsing captures semantic relationships through head-dependent structures
2. Natural language is globally ambiguous; parsers must resolve attachments
3. Transition-based parsing offers linear-time efficiency with greedy decisions
4. Neural networks provide:
   - Dense, distributed representations
   - Non-linear decision boundaries
   - Better generalization than sparse features
5. Modern parsers achieve ¿95% accuracy on standard benchmarks
6. Both transition-based and graph-based approaches are effective

Further Reading

- Chen & Manning (2014): "A Fast and Accurate Dependency Parser using Neural Networks"
- Dozat & Manning (2017): "Deep Biaffine Attention for Neural Dependency Parsing"
- Nivre (2003): "An Efficient Algorithm for Projective Dependency Parsing"
- Universal Dependencies documentation: `https://universaldependencies.org`